

The Complete SQL Interview Question Bank

Covers every core SQL topic (pre-PL/SQL) with detailed answers

Table of Contents

PART A — 226 THEORY QUESTIONS (TOPIC-WISE)

1. SQL & Database Fundamentals

Warm-up questions on what SQL and databases actually are.

Q1. What is SQL?

A: SQL (Structured Query Language) is the standard programming language used to store, manipulate, and retrieve data in a relational database. It lets you define database structures, insert and modify data, query data, and control access — all through declarative statements where you describe what you want rather than how to fetch it.

Note: SQL was developed at IBM in the 1970s and is now an ANSI/ISO standard that every relational database implements with small variations.

Q2. What is a database?

A: A database is an organized, electronic collection of related data stored so it can be efficiently accessed, managed, and updated. Instead of scattered files, data is held in a structured form (commonly tables) that supports searching, sorting, and linking of information.

Q3. What is a DBMS and what is an RDBMS?

A: A DBMS (Database Management System) is software that manages databases — it handles storage, retrieval, security, and concurrent access between users and the raw data. An RDBMS (Relational DBMS) is a specific type of DBMS that stores data in tables (relations) and links them using keys, following the relational model proposed by E.F. Codd.

Examples of RDBMS: MySQL, Oracle, PostgreSQL, SQL Server. All modern RDBMS use SQL as their query language.

Q4. What is the difference between SQL and MySQL?

A: SQL is a language used to query and manage relational databases, while MySQL is a specific RDBMS software product that implements SQL. In short, SQL is the language and MySQL is one of many tools that speak it.

Basis	SQL	MySQL
Type	Query language	Database software (RDBMS)
Purpose	Write queries, manage data	Store and administer databases
Standard	ANSI/ISO standard	A product owned by Oracle

Q5. What are the different sublanguages (command categories) of SQL?

A: SQL commands are grouped into five categories based on what they do:

Category	Meaning	Key Commands
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Data Manipulation Language	INSERT, UPDATE, DELETE
DQL	Data Query Language	SELECT
DCL	Data Control Language	GRANT, REVOKE
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT

Q6. What is the difference between DDL, DML, and DCL?

A: DDL defines or changes the structure of database objects (tables, schemas) and auto-commits. DML manipulates the data inside those objects (insert, update, delete) and can be rolled back. DCL controls permissions and access rights using GRANT and REVOKE.

Q7. What is a table, a row, and a column?

A: A table is a collection of related data organized into rows and columns and represents one entity (e.g., Employees).

- Row (record/tuple): one horizontal entry representing a single complete record.
- Column (field/attribute): one vertical entry holding a single type of data for every row.

Q8. What is a record?

A: A record is a single row in a table — a complete set of related field values describing one item, such as all details of one customer.

Q9. What is the difference between a field and a record?

A: A field is a single column value (one attribute, like 'City'), whereas a record is an entire row made up of many fields. Fields are vertical pieces of data; records are horizontal collections of those pieces.

Q10. What is a schema in SQL?

A: A schema is the logical structure or blueprint of a database. It defines the tables, the columns and their data types, relationships, constraints, views, and other objects, and acts as a namespace that groups related objects together.

Q11. What are the most common data types in SQL?

A: Data types define what kind of value a column can hold. Common groups:

Category	Examples	Use
Numeric	INT, BIGINT, DECIMAL, FLOAT	Numbers, money, measurements
String	CHAR, VARCHAR, TEXT	Names, descriptions
Date/Time	DATE, TIME, DATETIME, TIMESTAMP	Dates and times
Boolean/Other	BOOLEAN, BLOB	Flags, binary data

Q12. What is the difference between CHAR and VARCHAR?

A: CHAR is a fixed-length string type: it always reserves the declared length and pads shorter values with spaces. VARCHAR is variable-length: it stores only the characters entered plus a small length marker, so it saves space for values of differing length.

Note: Use CHAR for fixed-size codes (e.g., country code 'IN'), VARCHAR for variable text like names.

Q13. What is the difference between NULL, zero, and an empty string?

A: NULL means a value is missing or unknown — no data has been entered. Zero is a definite numeric value, and an empty string ("") is a defined text value of length zero. NULL is the absence of any value, so comparisons with it use IS NULL rather than = .

Q14. What is a NULL value and how do you test for it?

A: A NULL represents missing or unknown data. Because NULL is not equal to anything (not even another NULL), you cannot use = or <> to test it. Use IS NULL and IS NOT NULL instead.

```
SELECT * FROM Employees WHERE ManagerID IS NULL;
```

Q15. What is a query?

A: A query is a request written in SQL to retrieve or act on data in a database. Most queries are SELECT statements that read data, but the term broadly covers any SQL instruction sent to the database.

2. SELECT, Filtering & Sorting

The reading half of SQL — retrieving and shaping result sets.

Q16. What does the SELECT statement do?

A: SELECT retrieves data from one or more tables and returns it as a result set. It is the foundation of all data querying and can include filtering, grouping, sorting, joining, and calculations.

```
SELECT Name, Salary FROM Employees WHERE DeptID = 10;
```

Q17. What is the difference between SELECT * and SELECT specific columns?

A: SELECT * returns every column of the table, which is convenient but can return unnecessary data and hurt performance. Selecting specific columns returns only what you need, making queries faster, clearer, and less likely to break if the table structure changes.

Q18. What is the WHERE clause?

A: WHERE filters rows, returning only those that satisfy a given condition. It operates on individual rows before any grouping happens and cannot contain aggregate functions.

Q19. What is the difference between WHERE and HAVING?

A: WHERE filters individual rows before grouping and cannot use aggregate functions. HAVING filters groups after GROUP BY has been applied and is specifically used to filter on aggregate results.

```
SELECT DeptID, AVG(Salary) AS AvgSal  
FROM Employees  
WHERE Status = 'Active' -- filters rows first  
GROUP BY DeptID  
HAVING AVG(Salary) > 50000; -- filters groups after
```

Q20. What is the ORDER BY clause?

A: ORDER BY sorts the result set by one or more columns. The default is ascending (ASC); use DESC for descending. You can sort by multiple columns and even by column position or alias.

```
SELECT Name, Salary FROM Employees  
ORDER BY Salary DESC, Name ASC;
```

Q21. What is the GROUP BY clause?

A: GROUP BY collapses rows that share the same values in the listed columns into a single summary row per group. It is almost always used with aggregate functions to compute per-group totals, counts, or averages.

Q22. What is the DISTINCT keyword?

A: DISTINCT removes duplicate rows from a result set, returning only unique combinations of the selected columns.

```
SELECT DISTINCT City FROM Customers;
```

Q23. What is the difference between DISTINCT and GROUP BY?

A: Both can produce unique values. DISTINCT simply removes duplicate rows, while GROUP BY groups rows so aggregate functions can be applied per group. If you only need unique values with no aggregation, DISTINCT is simpler; if you need counts/sums per group, use GROUP BY.

Q24. What is the LIMIT clause (and its equivalents)?

A: LIMIT restricts how many rows are returned. Dialects differ: MySQL/PostgreSQL use LIMIT, SQL Server uses TOP, and Oracle uses FETCH FIRST n ROWS ONLY.

```
-- MySQL/PostgreSQL  
SELECT * FROM Employees ORDER BY Salary DESC LIMIT 5;  
-- SQL Server  
SELECT TOP 5 * FROM Employees ORDER BY Salary DESC;  
-- Oracle  
SELECT * FROM Employees ORDER BY Salary DESC FETCH FIRST 5 ROWS ONLY;
```

Q25. What is the use of OFFSET?

A: OFFSET skips a number of rows before starting to return rows, which together with LIMIT enables pagination.

```
-- skip 10 rows, then return the next 10 (page 2)  
SELECT * FROM Products ORDER BY ID LIMIT 10 OFFSET 10;
```

Q26. What is an alias in SQL?

A: An alias is a temporary, readable name given to a column or table within a query using AS. Column aliases rename output headers; table aliases shorten references, especially in joins.

```
SELECT e.Name AS Employee, d.Name AS Department  
FROM Employees AS e JOIN Departments AS d ON e.DeptID = d.ID;
```

Q27. What is the logical order of execution of a SELECT query?

A: Although we write SELECT first, the database logically processes clauses in this order:

```
FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT
```

This is why column aliases defined in SELECT generally cannot be used in WHERE (which runs earlier) but can be used in ORDER BY (which runs later).

3. Operators & Conditions

Q28. What are the main categories of SQL operators?

A: SQL operators fall into several groups:

- Arithmetic: +, -, *, /, %
- Comparison: =, <>, !=, >, <, >=, <=
- Logical: AND, OR, NOT
- Special: BETWEEN, IN, LIKE, IS NULL, EXISTS, ANY, ALL

Q29. What do the logical operators AND, OR, and NOT do?

A: AND returns true only when all conditions are true. OR returns true when at least one condition is true. NOT reverses the result of a condition. They combine multiple conditions in a WHERE clause.

Q30. What is the BETWEEN operator?

A: BETWEEN tests whether a value lies within a range, inclusive of both endpoints. It works with numbers, dates, and text.

```
SELECT * FROM Orders WHERE OrderDate BETWEEN '2024-01-01' AND '2024-03-31';
```

Q31. What is the IN operator?

A: IN checks whether a value matches any value in a given list or subquery result. It is shorthand for multiple OR conditions.

```
SELECT * FROM Employees WHERE DeptID IN (10, 20, 30);
```

Q32. What is the LIKE operator and what wildcards does it use?

A: LIKE performs pattern matching on strings. It uses two wildcards: % matches any number of characters (including zero), and _ matches exactly one character.

```
SELECT * FROM Customers WHERE Name LIKE 'A%'; -- starts with A
SELECT * FROM Customers WHERE Name LIKE '_a%'; -- second letter is a
```

Q33. What is the difference between = and LIKE?

A: = checks for an exact, full match of the value, while LIKE checks for a pattern and supports wildcards (% and _). Use = for exact comparisons and LIKE when matching partial text.

Q34. What is the difference between IN and EXISTS?

A: IN compares a value against a list or the results of a subquery. EXISTS checks whether a subquery returns any rows at all, returning true/false. EXISTS often performs better with large correlated subqueries because it stops at the first match, whereas IN may evaluate the full list.

```
-- EXISTS
SELECT Name FROM Customers c
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.CustID = c.ID);
```

Q35. What is the difference between IN and BETWEEN?

A: IN matches a value against a discrete list of specific values, while BETWEEN matches any value within a continuous range. Use IN for exact set membership and BETWEEN for ranges.

Q36. What are ANY and ALL operators?

A: ANY and ALL compare a value to a set returned by a subquery. ANY is true if the comparison holds for at least one value in the set; ALL is true only if it holds for every value.

```
-- salary greater than every salary in dept 10
SELECT Name FROM Employees
WHERE Salary > ALL (SELECT Salary FROM Employees WHERE DeptID = 10);
```

Q37. How do you negate conditions (NOT IN, NOT LIKE, NOT BETWEEN)?

A: Prefix the operator with NOT to exclude matches. Be careful with NOT IN when the subquery can return NULLs, because a NULL in the list makes the whole NOT IN result unknown (no rows).

```
SELECT * FROM Employees WHERE DeptID NOT IN (10, 20);
```

Q38. How is operator precedence handled in SQL?

A: SQL evaluates operators in a defined order: arithmetic first, then comparison, then NOT, then AND, then OR. Because AND binds tighter than OR, you should use parentheses to make combined conditions explicit and avoid logic errors.

4. SQL Functions (String, Numeric, Date)

Built-in functions that transform individual values.

Q39. What is the difference between scalar functions and aggregate functions?

A: A scalar function operates on a single value and returns one value per row (e.g., UPPER, ROUND, LEN). An aggregate function operates on a set of rows and returns a single summarized value for the group (e.g., SUM, AVG, COUNT).

Note: *Scalar = row-by-row; Aggregate = many rows collapsed into one.*

Q40. Name common string functions and what they do.

Function	Purpose
UPPER / LOWER	Change case
LENGTH / LEN	Number of characters
SUBSTRING	Extract part of a string
CONCAT	Join strings together
TRIM / LTRIM / RTRIM	Remove spaces
REPLACE	Replace text within a string
INSTR / CHARINDEX	Find position of a substring

Q41. What does the CONCAT function do?

A: CONCAT joins two or more strings into one. Many databases also support the || operator (Oracle/PostgreSQL) or + (SQL Server) for concatenation.

```
SELECT CONCAT(FirstName, ' ', LastName) AS FullName FROM Employees;
```

Q42. What does SUBSTRING do?

A: SUBSTRING extracts a portion of a string starting at a given position for a given length.

```
SELECT SUBSTRING('Database', 1, 4); -- returns 'Data'
```

Q43. Name common numeric functions.

A: Common numeric functions include ROUND (round to decimals), CEIL/CEILING (round up), FLOOR (round down), ABS (absolute value), MOD (remainder), and POWER (exponent). They transform numeric values row by row.

Q44. What does the ROUND function do?

A: ROUND rounds a number to a specified number of decimal places.


```
SELECT ROUND(123.4567, 2); -- returns 123.46
```

Q45. Name common date/time functions.

A: Date functions read or manipulate dates and times:

Function	Purpose
NOW() / GETDATE()	Current date and time
CURDATE() / CURRENT_DATE	Current date only
DATEDIFF	Difference between two dates
DATE_ADD / DATEADD	Add an interval to a date
YEAR(), MONTH(), DAY()	Extract a date part
EXTRACT	Extract any part of a date

Q46. How do you calculate the difference between two dates?

A: Use DATEDIFF. In MySQL it returns the number of days between two dates; SQL Server's DATEDIFF takes a unit argument (day, month, year).

```
-- MySQL: days between
SELECT DATEDIFF('2024-12-31', '2024-01-01'); -- 365
```

Q47. What is the COALESCE function?

A: COALESCE returns the first non-NULL value from a list of expressions. It is commonly used to substitute a default when a column may be NULL.

```
SELECT Name, COALESCE(Phone, 'N/A') AS Contact FROM Customers;
```

Q48. What is the difference between COALESCE, ISNULL, IFNULL, and NVL?

A: All replace NULLs, but they are dialect-specific and differ in flexibility:

Function	Database	Arguments
COALESCE	Standard (all)	Two or more; returns first non-NULL
IFNULL	MySQL	Exactly two
ISNULL	SQL Server	Exactly two
NVL	Oracle	Exactly two

Q49. What is the difference between NULLIF and COALESCE?

A: COALESCE returns the first non-NULL value among its arguments. NULLIF does the opposite kind of job: it returns NULL if its two arguments are equal, otherwise it returns the first argument — useful for avoiding division-by-zero.

Q50. What is the CASE expression?

A: CASE adds if-then-else logic inside a query, returning different values based on conditions. It is used in SELECT, WHERE, ORDER BY, and aggregates.

```
SELECT Name,
CASE WHEN Salary >= 80000 THEN 'High'
     WHEN Salary >= 50000 THEN 'Medium'
     ELSE 'Low' END AS Band
FROM Employees;
```

Q51. What is the difference between CAST and CONVERT?

A: Both change a value from one data type to another. CAST is ANSI-standard and works across databases, while CONVERT is dialect-specific (SQL Server) and offers extra formatting options such as date styles. Prefer CAST for portability.

5. Aggregate Functions & Grouping

Q52. What are aggregate functions?

A: Aggregate functions perform a calculation across a set of rows and return a single value. The five core ones are COUNT, SUM, AVG, MIN, and MAX. They are typically used with GROUP BY to summarize data per group.

Note: Aggregate functions (except COUNT(*)) ignore NULL values.

Q53. What is the difference between COUNT(*), COUNT(column), and COUNT(DISTINCT column)?

A: These count differently:

- COUNT(*) counts all rows, including those with NULLs.
- COUNT(column) counts only rows where that column is not NULL.
- COUNT(DISTINCT column) counts only the unique non-NULL values.

Q54. What is the difference between SUM and COUNT?

A: SUM adds up the numeric values in a column and returns the total, while COUNT returns the number of rows or non-NULL values. SUM works on the values; COUNT works on the number of items.

Q55. Can you use aggregate functions in the WHERE clause?

A: No. WHERE filters rows before grouping, so aggregates are not yet computed. To filter on an aggregate result, use HAVING after GROUP BY.

```
-- correct way to filter on an aggregate
SELECT DeptID, COUNT(*) AS Cnt
FROM Employees GROUP BY DeptID
HAVING COUNT(*) > 5;
```

Q56. How do aggregate functions handle NULL values?

A: Most aggregate functions ignore NULLs entirely. For example, AVG computes the average of non-NULL values only, and SUM skips NULLs. The exception is COUNT(*), which counts every row regardless of NULLs.

Q57. What columns can appear in SELECT when using GROUP BY?

A: Every non-aggregated column in the SELECT list must appear in the GROUP BY clause. Otherwise the database cannot decide which value of that column to show for the group (strict SQL modes reject such queries).

Q58. What is the difference between GROUP BY and ORDER BY?

A: GROUP BY groups rows into summary groups for aggregation, changing the number of rows returned. ORDER BY only sorts the final result set and does not change how many rows are returned.

Q59. What is the WITH ROLLUP / GROUPING SETS feature?

A: ROLLUP and GROUPING SETS extend GROUP BY to produce subtotals and grand totals in a single query. ROLLUP adds aggregated summary rows across the grouped columns, useful for reports that need totals at multiple levels.

Q60. How would you find the average salary per department, only for departments with more than 3 employees?


```
SELECT DeptID, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY DeptID
HAVING COUNT(*) > 3;
```

6. Keys

Keys uniquely identify rows and connect tables — a heavily tested topic.

Q61. What is a key in SQL?

A: A key is a column or set of columns used to uniquely identify rows in a table and to establish relationships between tables. Keys enforce uniqueness and integrity in the relational model.

Q62. What is a Primary Key?

A: A primary key uniquely identifies each row in a table. It must contain unique values and cannot contain NULLs. A table can have only one primary key, which may be a single column or a combination of columns.

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50)
);
```

Q63. What is a Foreign Key?

A: A foreign key is a column (or set of columns) in one table that references the primary key of another table. It enforces referential integrity, ensuring that the value exists in the parent table and preventing orphan records.

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustID INT,
    FOREIGN KEY (CustID) REFERENCES Customers(CustID)
);
```

Q64. What is a Unique Key?

A: A unique key ensures all values in a column (or column set) are distinct. Unlike a primary key, it allows one NULL value, and a table can have multiple unique keys.

Q65. What is the difference between a Primary Key and a Unique Key?

Basis	Primary Key	Unique Key
NULLs	Not allowed	One NULL allowed
Count per table	Only one	Many allowed
Purpose	Main row identifier	Enforce uniqueness on other columns
Default index	Clustered (often)	Non-clustered (often)

Q66. What is a Candidate Key?

A: A candidate key is any column or combination of columns that can uniquely identify a row and could therefore serve as the primary key. From all candidate keys, one is chosen as the primary key and the rest become alternate keys.

Q67. What is an Alternate Key?

A: An alternate key is a candidate key that was not selected as the primary key. It still uniquely identifies rows but is not the table's main identifier.

Q68. What is a Composite Key?

A: A composite key is a primary or unique key made up of two or more columns combined, used when no single column can uniquely identify a row.

```
CREATE TABLE Enrollment (
  StudentID INT,
  CourseID INT,
  PRIMARY KEY (StudentID, CourseID)
);
```

Q69. What is a Super Key?

A: A super key is any set of columns that uniquely identifies rows in a table. Every candidate key is a super key, but a super key may include extra, redundant columns that are not needed for uniqueness.

Q70. What is the relationship between super key, candidate key, and primary key?

A: A super key is any uniquely-identifying column set. A candidate key is a minimal super key (no redundant columns). A primary key is the one candidate key chosen as the table's main identifier. So: primary key $\subseteq$ candidate keys $\subseteq$ super keys.

Q71. What is a Surrogate Key?

A: A surrogate key is an artificial, system-generated identifier (often an auto-increment integer) used as the primary key instead of natural business data. It is stable, simple, and avoids issues when natural values change.

Q72. What is referential integrity?

A: Referential integrity is the rule that a foreign key value must always match an existing primary key value in the referenced table, or be NULL. It prevents orphan records and is enforced automatically by foreign key constraints.

Q73. What are ON DELETE CASCADE and ON UPDATE CASCADE?

A: These are referential actions on foreign keys. ON DELETE CASCADE automatically deletes child rows when the parent row is deleted; ON UPDATE CASCADE updates child foreign key values when the parent key changes. Other options include SET NULL and RESTRICT.

```
FOREIGN KEY (CustID) REFERENCES Customers(CustID)
ON DELETE CASCADE ON UPDATE CASCADE;
```

7. Constraints

Q74. What is a constraint?

A: A constraint is a rule applied to a column or table that restricts the data allowed, ensuring accuracy and integrity. Constraints are enforced automatically by the database whenever data is inserted or updated.

Q75. What are the main types of constraints in SQL?

Constraint	Purpose
NOT NULL	Column cannot store NULL
UNIQUE	All values must be different
PRIMARY KEY	Unique + NOT NULL row identifier
FOREIGN KEY	Enforces a link to another table
CHECK	Values must satisfy a condition
DEFAULT	Provides a value when none is given

Q76. What is the NOT NULL constraint?

A: NOT NULL ensures that a column must always contain a value and cannot be left empty (NULL). It is used for mandatory fields such as a name or an ID.

Q77. What is the CHECK constraint?

A: CHECK restricts the values allowed in a column by enforcing a Boolean condition. Any insert or update that violates the condition is rejected.

```
CREATE TABLE Employees (  
    ID INT,  
    Age INT CHECK (Age >= 18)  
);
```

Q78. What is the DEFAULT constraint?

A: DEFAULT assigns a predefined value to a column when no value is provided during an INSERT.

```
CREATE TABLE Orders (  
    ID INT,  
    Status VARCHAR(20) DEFAULT 'Pending'  
);
```

Q79. What is the difference between a column-level and a table-level constraint?

A: A column-level constraint is defined next to a single column and applies only to it. A table-level constraint is defined separately and can span multiple columns — required for composite primary keys or multi-column CHECK rules.

Q80. Can you add a constraint to an existing table?

A: Yes, using ALTER TABLE ... ADD CONSTRAINT. You can also drop constraints later by name.

```
ALTER TABLE Employees  
ADD CONSTRAINT chk_age CHECK (Age >= 18);
```

Q81. What is the difference between a UNIQUE constraint and a UNIQUE index?

A: A UNIQUE constraint is a logical rule that enforces no duplicate values; the database implements it by automatically creating a unique index behind the scenes. Functionally they enforce the same thing, but a constraint expresses intent and can be referenced by name, while an index is a physical structure.

8. Joins

Combining data across tables — the single most important practical topic.

Q82. What is a JOIN?

A: A JOIN combines rows from two or more tables based on a related column between them. It lets you query data that is spread across multiple normalized tables as if it were one combined result.

Q83. What are the different types of joins?

Join	Returns
INNER JOIN	Only matching rows in both tables
LEFT (OUTER) JOIN	All left rows + matching right rows
RIGHT (OUTER) JOIN	All right rows + matching left rows
FULL (OUTER) JOIN	All rows from both tables
CROSS JOIN	Cartesian product (every combination)

SELF JOIN

A table joined to itself

Q84. What is an INNER JOIN?

A: INNER JOIN returns only the rows that have matching values in both tables; unmatched rows are excluded.

```
SELECT e.Name, d.DeptName
FROM Employees e
INNER JOIN Departments d ON e.DeptID = d.DeptID;
```

Q85. What is a LEFT JOIN?

A: LEFT JOIN (LEFT OUTER JOIN) returns all rows from the left table and the matching rows from the right table. Where there is no match, the right-table columns are filled with NULL.

```
SELECT c.Name, o.OrderID
FROM Customers c
LEFT JOIN Orders o ON c.ID = o.CustID; -- includes customers with no orders
```

Q86. What is a RIGHT JOIN?

A: RIGHT JOIN (RIGHT OUTER JOIN) returns all rows from the right table plus the matching rows from the left table, filling unmatched left columns with NULL. It is the mirror image of LEFT JOIN.

Q87. What is a FULL OUTER JOIN?

A: FULL OUTER JOIN returns all rows from both tables. Rows that match are combined; rows with no match on either side appear with NULLs for the missing columns.

Note: MySQL has no native FULL OUTER JOIN; you emulate it with LEFT JOIN UNION RIGHT JOIN.

Q88. What is a CROSS JOIN?

A: A CROSS JOIN produces the Cartesian product of two tables — each row of the first is combined with every row of the second. If the tables have m and n rows, the result has m × n rows. It has no ON clause.

```
SELECT s.Size, c.Color FROM Sizes s CROSS JOIN Colors c;
```

Q89. What is a SELF JOIN?

A: A SELF JOIN joins a table to itself by using table aliases. It is useful for comparing rows within the same table, such as matching employees to their managers.

```
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employees e
JOIN Employees m ON e.ManagerID = m.EmpID;
```

Q90. What is the difference between INNER JOIN and OUTER JOIN?

A: INNER JOIN returns only rows with matches in both tables. OUTER JOIN (LEFT, RIGHT, FULL) returns matching rows plus the unmatched rows from one or both tables, using NULLs to fill the gaps. Use INNER when you need only related data, OUTER when you must keep unmatched rows.

Q91. What is the difference between a JOIN and a UNION?

A: A JOIN combines columns from different tables side by side based on a related condition (widens the result). A UNION stacks the rows of two result sets on top of each other (lengthens the result) and requires both queries to have the same number and types of columns.

Q92. What is the ON clause versus the USING clause in joins?

A: ON specifies an explicit join condition and is the most flexible. USING is a shorthand when both tables share a column of the same name, joining on that column and merging it into one output column.

```
SELECT * FROM Orders JOIN Customers USING (CustID);
```

Q93. What happens if you join tables without a join condition?

A: Omitting the join condition produces a Cartesian product (a cross join), matching every row of one table with every row of the other. This usually returns a huge, meaningless result and is a common accidental mistake.

Q94. Can you join more than two tables?

A: Yes. You can chain multiple JOINS in a single query, each with its own ON condition, to combine data from many tables.

```
SELECT o.OrderID, c.Name, p.ProductName
FROM Orders o
JOIN Customers c ON o.CustID = c.ID
JOIN Products p ON o.ProductID = p.ID;
```

Q95. What is the difference between an equi join and a non-equi join?

A: An equi join uses equality (=) in its join condition, which is the most common case. A non-equi join uses other operators such as <, >, or BETWEEN — for example, joining a sales amount to a commission-bracket table by range.

Q96. What is a natural join?

A: A NATURAL JOIN automatically joins two tables on all columns that share the same name, without an explicit ON clause. It is concise but risky, because adding a same-named column later can silently change the join, so explicit ON is usually preferred.

9. Subqueries

Q97. What is a subquery?

A: A subquery is a query nested inside another SQL statement. The inner query runs and its result is used by the outer query. Subqueries can appear in SELECT, FROM, WHERE, and HAVING clauses.

```
SELECT Name FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

Q98. What are the types of subqueries?

- Scalar subquery: returns a single value.
- Row subquery: returns a single row of multiple columns.
- Table (multi-row) subquery: returns multiple rows, used with IN/ANY/ALL.
- Correlated subquery: references the outer query and runs per outer row.

Q99. What is the difference between a correlated and a non-correlated subquery?

A: A non-correlated subquery is independent — it runs once and its result is passed to the outer query. A correlated subquery references columns from the outer query, so it is evaluated once for every row processed by the outer query, making it potentially slower.

```
-- correlated: inner query depends on outer row e2
SELECT e1.Name FROM Employees e1
WHERE Salary > (SELECT AVG(Salary) FROM Employees e2
               WHERE e2.DeptID = e1.DeptID);
```

Q100. What is a subquery in the FROM clause (derived table)?

A: A subquery in the FROM clause acts as a temporary table, called a derived table or inline view, that the outer query reads from. It must be given an alias.

```
SELECT DeptID, MaxSal
FROM (SELECT DeptID, MAX(Salary) AS MaxSal
      FROM Employees GROUP BY DeptID) AS dept_max;
```

Q101. What is the difference between a subquery and a join?

A: Both combine data from multiple tables. A join merges rows side by side and is usually more efficient for retrieving columns from several tables. A subquery nests one query inside another and is clearer for filtering by an aggregated or derived value. Many problems can be solved either way; joins often perform better.

Q102. Can a subquery return multiple rows? How do you handle it?

A: Yes. A subquery used with =, <, or > must return a single value, but when it returns multiple rows you must use IN, ANY, ALL, or EXISTS to compare against the set, otherwise the query errors out.

Q103. What is the EXISTS operator and when is it used with subqueries?

A: EXISTS returns true if the correlated subquery produces at least one row, and is used to test for the presence of related records. It short-circuits at the first match, which makes it efficient.

```
SELECT Name FROM Customers c
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.CustID = c.ID);
```

10. Set Operations (UNION, INTERSECT, EXCEPT)

Q104. What are set operators in SQL?

A: Set operators combine the results of two or more SELECT queries into a single result set. The main ones are UNION, UNION ALL, INTERSECT, and EXCEPT/MINUS. All require the queries to have the same number of columns with compatible data types.

Q105. What is the difference between UNION and UNION ALL?

A: UNION combines two result sets and removes duplicate rows, which requires an extra sorting/deduplication step. UNION ALL combines them and keeps all rows including duplicates, making it faster. Use UNION ALL when you know there are no duplicates or want to keep them.

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers;
```

Q106. What is INTERSECT?

A: INTERSECT returns only the rows that appear in both query results — the common rows. Like UNION, it removes duplicates from the final output.

Q107. What is EXCEPT (or MINUS)?

A: EXCEPT (called MINUS in Oracle) returns rows from the first query that do not appear in the second query — the difference between the two sets.

```
SELECT ID FROM AllCustomers
EXCEPT
SELECT ID FROM CustomersWithOrders; -- customers with no orders
```

Q108. What are the rules for using set operators?

A: All queries combined by set operators must satisfy these rules:

- Same number of columns in each SELECT.
- Corresponding columns must have compatible data types.
- Column names in the output come from the first query.
- ORDER BY, if used, goes once at the very end.

Q109. What is the difference between UNION and JOIN?

A: UNION combines results vertically by stacking rows from queries with the same columns. JOIN combines tables horizontally by matching related columns, widening each row. UNION adds rows; JOIN adds columns.

11. Views

Q110. What is a view?

A: A view is a virtual table defined by a stored SELECT query. It does not store data itself; instead it presents data from one or more underlying tables each time it is queried. Views simplify complex queries, provide a security layer, and present a consistent interface.

```
CREATE VIEW ActiveCustomers AS  
SELECT ID, Name, City FROM Customers WHERE Status = 'Active';
```

Q111. What are the advantages of using views?

- Simplify complex queries by hiding joins and logic behind a name.
- Improve security by exposing only certain columns/rows.
- Provide data independence — underlying tables can change while the view's interface stays stable.
- Promote reusability of common query logic.

Q112. What is the difference between a view and a table?

A: A table physically stores data on disk, while a view stores only a query definition and generates its data on the fly when queried. Tables hold data; views present it.

Q113. Can you update data through a view?

A: Sometimes. A simple view based on a single table without aggregates, DISTINCT, GROUP BY, or joins is usually updatable, and changes pass through to the base table. Complex views involving joins, grouping, or functions are generally read-only.

Q114. What is a materialized view?

A: A materialized view physically stores the result of its query on disk, unlike a normal view which is virtual. It speeds up expensive queries by caching results, but the data can become stale and must be refreshed periodically.

Note: Supported in Oracle and PostgreSQL; MySQL has no native materialized views.

Q115. What is the difference between a view and a materialized view?

A: A normal view runs its query every time it is accessed and always shows current data, but adds computation each time. A materialized view stores precomputed results for fast reads, but needs refreshing and may show slightly outdated data. It is a trade-off between freshness and speed.

12. Indexes

Q116. What is an index?

A: An index is a database object that improves the speed of data retrieval on a table, much like the index at the back of a book. It stores a sorted structure (often a B-tree) of column values pointing to the matching rows, so the database can find data without scanning every row.

```
CREATE INDEX idx_lastname ON Employees(LastName);
```

Q117. What are the advantages and disadvantages of indexes?

A: Advantages: faster SELECT and WHERE lookups, faster sorting and joining on indexed columns. Disadvantages: indexes consume extra storage and slow down INSERT, UPDATE, and DELETE because the index must also be maintained. So you index selectively, on columns used frequently in searches and joins.

Q118. What is the difference between a clustered and a non-clustered index?

Aspect	Clustered	Non-clustered
Row order	Defines physical order of rows	Separate structure with pointers
Number per table	Only one	Many allowed
Speed	Faster for range scans	Extra lookup to fetch row
Analogy	Dictionary (data sorted itself)	Book index (points to pages)

Q119. What is a composite (multi-column) index?

A: A composite index is built on two or more columns together. It speeds up queries that filter or sort by those columns in order. The column order matters: an index on (A, B) helps queries on A or A+B, but not on B alone.

Q120. What is a unique index?

A: A unique index enforces that all values in the indexed column(s) are distinct, while also speeding up lookups. Creating a UNIQUE constraint or primary key automatically creates a unique index.

Q121. When should you avoid creating an index?

A: Avoid indexes on small tables, on columns with very low selectivity (few distinct values like a gender flag), and on columns that are rarely searched but frequently updated. Over-indexing wastes storage and slows down write operations.

Q122. What is a covering index?

A: A covering index is one that includes all the columns a query needs, so the database can answer the query from the index alone without reading the table. This avoids extra lookups and makes the query much faster.

13. Normalization & Database Design

Q123. What is normalization?

A: Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It involves splitting large tables into smaller, related tables and defining relationships, so each piece of data is stored only once.

Q124. What are the main normal forms?

Normal Form	Rule (simplified)
1NF	Atomic values; no repeating groups
2NF	1NF + no partial dependency on part of a composite key
3NF	2NF + no transitive dependency (non-key on non-key)
BCNF	Stronger 3NF; every determinant is a candidate key

Q125. What is First Normal Form (1NF)?

A: A table is in 1NF when every column holds atomic (indivisible) values and there are no repeating groups or arrays in a cell. For example, storing multiple phone numbers in one column violates 1NF.

Q126. What is Second Normal Form (2NF)?

A: A table is in 2NF when it is in 1NF and every non-key column depends on the whole primary key, not just part of it. This matters only for composite keys; it removes partial dependencies.

Q127. What is Third Normal Form (3NF)?

A: A table is in 3NF when it is in 2NF and has no transitive dependencies — non-key columns must depend only on the primary key, not on other non-key columns. For example, storing both ZipCode and City (where City depends on ZipCode) violates 3NF.

Q128. What is denormalization and why is it used?

A: Denormalization is the deliberate process of adding redundancy by combining tables or duplicating data to improve read performance. It reduces the number of joins needed for queries, at the cost of extra storage and more complex updates. It is common in reporting and data warehouses.

Q129. What are the advantages of normalization?

A: Normalization reduces data redundancy, prevents update/insert/delete anomalies, saves storage, and keeps data consistent because each fact is stored in only one place. The trade-off is that heavily normalized designs may need more joins to retrieve data.

Q130. What is the difference between OLTP and OLAP?

A: OLTP (Online Transaction Processing) systems handle many short read/write transactions and are highly normalized — used for day-to-day operations like banking. OLAP (Online Analytical Processing) systems handle complex analytical queries over large historical data and are often denormalized — used for reporting and business intelligence.

14. Transactions & Concurrency

Q131. What is a transaction?

A: A transaction is a sequence of one or more SQL operations executed as a single logical unit of work. Either all operations succeed and are committed, or none take effect and they are rolled back. A classic example is transferring money: debit one account and credit another must both happen or neither.

```
BEGIN;
UPDATE Accounts SET bal = bal - 100 WHERE id = 1;
UPDATE Accounts SET bal = bal + 100 WHERE id = 2;
COMMIT;
```

Q132. What are the ACID properties?

A: ACID guarantees reliable transactions:

Property	Meaning
Atomicity	All steps complete or none do
Consistency	Database moves from one valid state to another
Isolation	Concurrent transactions don't interfere
Durability	Committed changes survive crashes

Q133. What is the difference between COMMIT and ROLLBACK?

A: COMMIT permanently saves all changes made in the current transaction to the database. ROLLBACK undoes all changes since the transaction began (or since the last savepoint), returning the data to its previous state.

Q134. What is a SAVEPOINT?

A: A SAVEPOINT marks a point within a transaction to which you can later roll back, without undoing the entire transaction. It allows partial rollbacks of a larger transaction.

```
SAVEPOINT sp1;
-- ... some statements ...
ROLLBACK TO sp1; -- undo only back to sp1
```

Q135. What are the SQL transaction isolation levels?

A: Isolation levels control how much one transaction sees of others' uncommitted work, trading consistency against concurrency:

Level	Prevents
Read Uncommitted	Nothing (allows dirty reads)
Read Committed	Dirty reads
Repeatable Read	Dirty + non-repeatable reads
Serializable	Dirty, non-repeatable & phantom reads

Q136. What are dirty reads, non-repeatable reads, and phantom reads?

- Dirty read: reading data another transaction has written but not yet committed.
- Non-repeatable read: reading the same row twice and getting different values because another transaction updated it in between.
- Phantom read: re-running a query and seeing new rows that another transaction inserted.

Q137. What is a deadlock and how is it handled?

A: A deadlock occurs when two transactions each hold a lock the other needs, so both wait forever. Databases detect deadlocks automatically and resolve them by aborting one transaction (the 'victim'), which is then rolled back so the other can proceed. Applications should retry the aborted transaction.

Q138. What is the difference between optimistic and pessimistic locking?

A: Pessimistic locking locks rows as soon as they are read, assuming conflicts are likely, which prevents others from modifying them until the lock is released. Optimistic locking assumes conflicts are rare and does not lock; instead it checks at commit time (often using a version column) whether the data changed, and retries if it did.

15. Window Functions

Increasingly common in analyst interviews.

Q139. What is a window function?

A: A window function performs a calculation across a set of rows related to the current row (the 'window') without collapsing them into a single output row, unlike GROUP BY. It is defined using the OVER() clause.

```
SELECT Name, DeptID, Salary,
       AVG(Salary) OVER (PARTITION BY DeptID) AS DeptAvg
FROM Employees;
```

Q140. What is the difference between aggregate functions and window functions?

A: Aggregate functions with GROUP BY collapse many rows into one summary row per group. Window functions compute a value over a group of rows but keep every original row in the output, attaching the computed value to each row.

Q141. What is the difference between ROW_NUMBER(), RANK(), and DENSE_RANK()?

A: All assign numbers to rows within a partition, but they handle ties differently:

Value	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
100	2	1	1
90	3	3	2
80	4	4	3

ROW_NUMBER gives unique sequential numbers; RANK repeats numbers for ties and then skips; DENSE_RANK repeats numbers for ties without skipping.

Q142. What are PARTITION BY and ORDER BY in a window function?

A: PARTITION BY divides the rows into groups (windows), and the function restarts for each partition — like GROUP BY but without collapsing rows. ORDER BY within OVER() defines the order of rows inside each partition, which matters for ranking and running totals.

Q143. What are LAG() and LEAD()?

A: LAG() returns a value from a previous row, and LEAD() returns a value from a following row, within the partition. They are used to compare a row with its neighbors, such as month-over-month changes.

```
SELECT Month, Sales,
       LAG(Sales) OVER (ORDER BY Month) AS PrevSales
FROM MonthlySales;
```

Q144. What is a running total and how do you compute it?

A: A running (cumulative) total sums values up to and including the current row. It is computed with a windowed SUM ordered by the relevant column.

```
SELECT OrderDate, Amount,
       SUM(Amount) OVER (ORDER BY OrderDate) AS RunningTotal
FROM Orders;
```

Q145. What is the NTILE() function?

A: NTILE(n) divides the ordered rows of a partition into n roughly equal buckets and assigns each row its bucket number. It is used for percentiles and quartiles, such as splitting customers into four spending tiers.

16. Advanced & Miscellaneous Concepts

Q146. What is a CTE (Common Table Expression)?

A: A CTE is a temporary, named result set defined with the WITH keyword that exists only for the duration of a single query. It improves readability, can be referenced multiple times, and supports recursion.

```
WITH HighEarnings AS (
  SELECT * FROM Employees WHERE Salary > 90000
)
SELECT DeptID, COUNT(*) FROM HighEarnings GROUP BY DeptID;
```

Q147. What is the difference between a CTE and a subquery?

A: Both produce temporary result sets. A subquery is nested inline and can be harder to read when reused, while a CTE is defined once at the top with WITH, can be referenced multiple times in the same query, and supports recursion. CTEs generally improve readability.

Q148. What is a recursive CTE?

A: A recursive CTE references itself to process hierarchical or tree-structured data, such as organizational charts or bill-of-materials. It has an anchor member and a recursive member joined by UNION ALL.

```
WITH RECURSIVE OrgChart AS (
  SELECT EmpID, Name, ManagerID FROM Employees WHERE ManagerID IS NULL
  UNION ALL
  SELECT e.EmpID, e.Name, e.ManagerID
  FROM Employees e JOIN OrgChart o ON e.ManagerID = o.EmpID
)
SELECT * FROM OrgChart;
```

Q149. What is the difference between DELETE, TRUNCATE, and DROP?

Feature	DELETE	TRUNCATE	DROP
Removes	Selected/all rows	All rows	Entire table
WHERE clause	Yes	No	No
Rollback	Yes	Limited	No
Resets identity	No	Yes	N/A
Type	DML	DDL	DDL
Speed	Slow	Fast	Fast

Q150. What is the difference between a stored procedure and a function?

Aspect	Function	Stored Procedure
Return	Must return a value	May return zero or more values
Use in SELECT	Yes	No
DML (insert/update)	Generally not allowed	Allowed
Call	Inside a query	EXEC / CALL
Transactions	Cannot manage	Can manage

Q151. What is a trigger?

A: A trigger is a stored block of SQL that runs automatically in response to a data event — INSERT, UPDATE, or DELETE — on a table. Triggers enforce business rules, maintain audit logs, or keep derived data in sync.

```
CREATE TRIGGER trg_audit AFTER UPDATE ON Employees
FOR EACH ROW
INSERT INTO AuditLog(EmpID, ChangedAt) VALUES (OLD.EmpID, NOW());
```

Q152. What are the BEFORE and AFTER trigger types?

A: A BEFORE trigger fires before the triggering operation is applied, often used to validate or modify incoming data. An AFTER trigger fires after the operation completes, used for logging or cascading actions to other tables.

Q153. What is the difference between a procedure and a trigger?

A: A stored procedure is invoked explicitly by a user or application using CALL/EXEC. A trigger runs automatically and implicitly when a defined data event occurs on a table; it cannot be called directly.

Q154. What is the difference between TRUNCATE and DELETE in terms of logging and triggers?

A: DELETE removes rows one by one, logs each deletion, can use a WHERE clause, and fires row-level triggers. TRUNCATE deallocates whole data pages, uses minimal logging, cannot use WHERE, resets identity counters, and does not fire row-level DELETE triggers, making it much faster.

Q155. What is the difference between RANK and ROW_NUMBER (quick recap)?

A: ROW_NUMBER always assigns unique consecutive numbers even to tied values, whereas RANK gives tied rows the same number and then skips the following number(s). ROW_NUMBER is for unique ordering; RANK is for competition-style ranking.

Q156. What is a self-contained vs correlated subquery (quick recap)?

A: A self-contained (non-correlated) subquery can run on its own and is evaluated once. A correlated subquery depends on the outer query's current row and is evaluated repeatedly, once per outer row.

Q157. What is the difference between the HAVING and WHERE clause (quick recap)?

A: WHERE filters rows before grouping and cannot contain aggregates; HAVING filters groups after aggregation and is used specifically to test aggregate results like COUNT or SUM.

Q158. What is a primary difference between clustered and non-clustered indexes (quick recap)?

A: A clustered index determines the physical storage order of the table's rows and there can be only one. A non-clustered index is a separate structure that points back to the rows, and a table can have many of them.

Q159. What does the EXPLAIN / execution plan tell you?

A: EXPLAIN shows the query execution plan — how the database intends to run a query, including the join order, whether indexes are used, and estimated row counts. It is the primary tool for diagnosing and tuning slow queries.

Q160. What is SQL injection and how do you prevent it?

A: SQL injection is a security attack where malicious input is inserted into a query to manipulate or expose data. It is prevented by using parameterized queries / prepared statements, validating and sanitizing input, applying least-privilege database accounts, and avoiding string concatenation of user input into SQL.

Note: Prepared statements separate code from data, so user input can never be executed as SQL.

17. Window Functions — In-Depth

A deeper, interview-focused treatment of window (analytic) functions, frames, and comparisons.

Q161. What exactly is the OVER() clause, and what are its components?

A: OVER() defines the 'window' of rows a window function operates on. It has three optional parts: PARTITION BY (divides rows into groups), ORDER BY (orders rows inside each partition), and a frame specification (ROWS/RANGE BETWEEN ... that limits which rows around the current row are considered).

```
<function>() OVER (
  PARTITION BY col1
  ORDER BY col2
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)
```

Q162. What is the difference between window functions and GROUP BY?

A: Both summarize data, but they differ fundamentally in what they return:

Aspect	GROUP BY	Window Function
Rows returned	One per group (collapses rows)	All original rows preserved

Detail + aggregate	Cannot show both easily	Shows row detail and aggregate together
Ranking	Not possible directly	ROW_NUMBER/RANK/DENSE_RANK available
Running totals	Hard	Natural with ORDER BY in OVER
Where used	Whole-query grouping	Per-row analytic calculation

Note: Rule of thumb: use *GROUP BY* when you want summary rows; use a window function when you need the aggregate next to each detail row.

Q163. Can you use a window function and GROUP BY in the same query?

A: Yes. GROUP BY is applied first to form grouped rows, and the window function then operates on those already-grouped rows. This is useful for ranking aggregated results, such as ranking departments by their total salary.

```
SELECT DeptID, SUM(Salary) AS Total,
       RANK() OVER (ORDER BY SUM(Salary) DESC) AS DeptRank
FROM Employees
GROUP BY DeptID;
```

Q164. Why can't window functions be used in the WHERE clause?

A: Window functions are evaluated after WHERE, GROUP BY, and HAVING in the logical processing order — they run during the SELECT phase. Since WHERE runs earlier, the window result does not yet exist there. To filter on a window result, wrap the query in a subquery/CTE and filter in the outer query.

```
SELECT * FROM (
  SELECT Name, Salary,
         ROW_NUMBER() OVER (ORDER BY Salary DESC) AS rn
  FROM Employees
) t
WHERE rn <= 3; -- filter the window result here
```

Q165. What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER() in depth?

A: All three are ranking functions but treat ties differently:

Salary	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
100	2	1	1
90	3	3	2
90	4	3	2
80	5	5	3

ROW_NUMBER always produces unique numbers (ties broken arbitrarily). RANK gives ties the same rank, then leaves gaps (jumps from 1 to 3). DENSE_RANK gives ties the same rank with no gaps (1,1,2,2,3).

Q166. When would you choose ROW_NUMBER over RANK or DENSE_RANK?

A: Use ROW_NUMBER when you need a strictly unique sequence — for example, to pick exactly one row per group (deduplication, 'latest record per customer'). Use RANK/DENSE_RANK when ties should genuinely share a position, such as a leaderboard where two people can tie for first place.

Q167. What is a window frame (ROWS vs RANGE)?

A: A frame defines which rows within the partition the function actually uses, relative to the current row. ROWS counts physical rows, while RANGE works on logical value ranges (rows with the same ORDER BY value are treated together).

-- last 3 rows including current (physical)
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
-- all rows with values up to current (logical)
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

Q168. What is the difference between ROWS and RANGE in a frame, with an example?

A: With duplicate ORDER BY values, ROWS and RANGE diverge. ROWS treats each tied row separately; RANGE groups all tied rows together into the same frame boundary. So a running SUM with RANGE will give the same cumulative value to all rows sharing an ORDER BY value, while ROWS will increment row by row.

Note: If you don't specify a frame but you do specify ORDER BY, the default is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW — a common source of subtle bugs.

Q169. What is the default window frame when ORDER BY is present but no frame is specified?

A: The default is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. This means cumulative aggregates include all rows from the start of the partition up to the current row's value group. If you want a precise row-by-row running total, specify ROWS explicitly to avoid ties being lumped together.

Q170. What do UNBOUNDED PRECEDING, CURRENT ROW, and UNBOUNDED FOLLOWING mean?

- UNBOUNDED PRECEDING: start of the partition (first row).
- CURRENT ROW: the row being evaluated.
- UNBOUNDED FOLLOWING: end of the partition (last row).
- n PRECEDING / n FOLLOWING: n rows before/after the current row.

Q171. What is the difference between LAG() and LEAD()?

A: LAG() looks backward to a previous row, while LEAD() looks forward to a following row, within the ordered partition. Both accept an optional offset (default 1) and a default value for when no such row exists.

```
SELECT Month, Sales,  
       LAG(Sales,1,0) OVER (ORDER BY Month) AS PrevMonth,  
       LEAD(Sales,1,0) OVER (ORDER BY Month) AS NextMonth  
FROM MonthlySales;
```

Q172. What is the difference between FIRST_VALUE() and LAST_VALUE()?

A: FIRST_VALUE() returns the first value in the window frame and LAST_VALUE() returns the last. A common pitfall: LAST_VALUE often looks 'wrong' because the default frame ends at CURRENT ROW, so it returns the current row instead of the true last row. Fix it by extending the frame.

```
LAST_VALUE(Salary) OVER (  
  PARTITION BY DeptID ORDER BY Salary  
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
)
```

Q173. What is NTH_VALUE() used for?

A: NTH_VALUE(expr, n) returns the value of expr from the nth row of the window frame — for example, the 2nd highest salary within each department. Like LAST_VALUE, it is sensitive to the frame definition.

```
NTH_VALUE(Salary, 2) OVER (  
  PARTITION BY DeptID ORDER BY Salary DESC  
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
)
```

Q174. What is the NTILE() function and when is it used?

A: NTILE(n) splits the ordered rows of each partition into n approximately equal buckets and labels each row with its bucket number (1..n). It is used for percentile-style bucketing — quartiles, deciles, customer tiers — when you want roughly equal-sized groups.

Q175. What is the difference between PERCENT_RANK() and CUME_DIST()?

A: Both express relative standing as a fraction between 0 and 1. PERCENT_RANK = (rank - 1) / (total rows - 1), showing the relative rank position. CUME_DIST (cumulative distribution) = (number of rows with value <= current) / total rows, showing the proportion of rows at or below the current value.

Note: PERCENT_RANK starts at 0 for the first row; CUME_DIST never returns 0.

Q176. How do you compute a running (cumulative) total?

A: Use a windowed SUM with ORDER BY and an explicit ROWS frame for correctness:

```
SELECT OrderDate, Amount,  
       SUM(Amount) OVER (ORDER BY OrderDate  
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal  
FROM Orders;
```

Q177. How do you compute a moving average over the last N rows?

A: Combine AVG with a sliding frame of N rows ending at the current row:

```
SELECT Day, Sales,  
       AVG(Sales) OVER (ORDER BY Day  
                        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS MovingAvg7  
FROM DailySales;
```

Q178. How do window functions interact with PARTITION BY versus a GROUP BY query?

A: PARTITION BY resets the window calculation for each partition but keeps every row, whereas GROUP BY produces one row per group. PARTITION BY is essentially 'GROUP BY without collapsing' — you keep detail rows and still get per-group aggregates, rankings, or running totals.

Q179. Can you use multiple different window definitions in one SELECT?

A: Yes. Each window function can have its own OVER() clause, so a single query can mix a department ranking, a company-wide running total, and a moving average simultaneously. Repeated window definitions can be named once using a WINDOW clause for clarity.

```
SELECT Name, Salary,  
       RANK() OVER w AS DeptRank,  
       AVG(Salary) OVER w AS DeptAvg  
FROM Employees  
WINDOW w AS (PARTITION BY DeptID ORDER BY Salary DESC);
```

Q180. What is the difference between an aggregate used normally and the same aggregate used as a window function?

A: SUM(Salary) with GROUP BY returns one total per group. SUM(Salary) OVER (PARTITION BY DeptID) returns the same total but attached to every employee row in that department, leaving the individual rows intact. The function is identical; the OVER clause changes the behavior from collapsing to annotating.

Q181. How would you find the top N rows per group using window functions?

A: Rank within each partition, then filter in an outer query:

```
SELECT * FROM (  
  SELECT Name, DeptID, Salary,  
         ROW_NUMBER() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS rn  
  FROM Employees  
) t  
WHERE rn <= 3;
```

Q182. How do you remove duplicates using window functions?

A: Assign a row number within groups of duplicate keys and keep only rn = 1 (or delete rn > 1):

```
WITH d AS (  
  SELECT *, ROW_NUMBER() OVER  
    (PARTITION BY Email ORDER BY ID) AS rn  
  FROM Users
```

```
)  
DELETE FROM Users WHERE ID IN (SELECT ID FROM d WHERE rn > 1);
```

Q183. How do you find gaps in a sequence using LAG()?

A: Compare each value to the previous one; a gap exists where the difference is greater than 1:

```
SELECT PrevID + 1 AS GapStart, ID - 1 AS GapEnd  
FROM (  
    SELECT ID, LAG(ID) OVER (ORDER BY ID) AS PrevID FROM Tickets  
) t  
WHERE ID - PrevID > 1;
```

Q184. What are the 'islands and gaps' problems, and how do window functions help?

A: Islands-and-gaps problems involve finding consecutive runs (islands) or missing stretches (gaps) in ordered data — for example, consecutive login days. A common technique subtracts ROW_NUMBER() from the sequence value: rows in the same consecutive run share a constant difference, so grouping by that difference identifies each island.

Q185. Do window functions change the number of rows returned by a query?

A: No. Window functions never add or remove rows; they only add computed columns to the existing rows. This is their defining difference from GROUP BY, which reduces rows to one per group.

Q186. Are window functions evaluated before or after DISTINCT and ORDER BY?

A: Window functions are computed in the SELECT phase, after GROUP BY/HAVING but before DISTINCT and the final ORDER BY. This means DISTINCT operates on rows that already include the window results, and the outer ORDER BY can sort by a window-produced column.

Q187. What is the performance consideration with window functions?

A: Window functions usually require sorting the data by the PARTITION BY and ORDER BY columns, which can be expensive on large datasets. Performance improves when supporting indexes match the partition/order columns, and when frames are kept simple. They are still generally faster and cleaner than equivalent self-joins or correlated subqueries.

Q188. Can window functions be nested or used inside aggregate functions?

A: You cannot nest one window function directly inside another, nor place a window function inside an aggregate, in the same SELECT level. To layer them (e.g., rank a running total), compute the first in a subquery or CTE and apply the second in the outer query.

Q189. What is the difference between PARTITION BY and ORDER BY inside OVER()?

A: PARTITION BY divides rows into independent groups and the function restarts for each group, while ORDER BY arranges rows within each partition and is what makes ranking, LAG/LEAD, and running totals meaningful. PARTITION BY controls 'which rows belong together'; ORDER BY controls 'in what sequence'.

Q190. Give a real-world scenario where a window function is clearly better than GROUP BY.

A: Showing each employee's salary next to their department average and their rank within the department. With GROUP BY you would lose the individual rows or need a self-join; a window function delivers detail and analytics in one pass:

```
SELECT Name, DeptID, Salary,  
    AVG(Salary) OVER (PARTITION BY DeptID) AS DeptAvg,  
    RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS DeptRank  
FROM Employees;
```

18. CTEs, Views & Temporary Tables — In-Depth

Temporary result-set mechanisms and how to choose between them.

Q191. What is a CTE (Common Table Expression)?

A: A CTE is a named, temporary result set defined with the WITH keyword that exists only for the duration of the single statement that follows it. It improves readability by letting you break a complex query into logical, named building blocks, and it can be referenced multiple times within that statement.

```
WITH DeptAvg AS (
  SELECT DeptID, AVG(Salary) AS AvgSal
  FROM Employees GROUP BY DeptID
)
SELECT e.Name, e.Salary, d.AvgSal
FROM Employees e
JOIN DeptAvg d ON e.DeptID = d.DeptID;
```

Q192. What are the advantages of using a CTE?

- Readability: complex logic is split into named, top-to-bottom steps.
- Reusability: the CTE can be referenced multiple times in the same query.
- Recursion: CTEs can call themselves to walk hierarchies.
- No object created: unlike a view or temp table, nothing persists in the schema.

Q193. What is a recursive CTE and how does it work?

A: A recursive CTE references itself to process hierarchical or sequential data. It has an anchor member (the starting rows) and a recursive member (which repeatedly joins back to the CTE), combined with UNION ALL. Recursion stops when the recursive member returns no new rows.

```
WITH RECURSIVE OrgChart AS (
  SELECT EmpID, Name, ManagerID, 1 AS Lvl
  FROM Employees WHERE ManagerID IS NULL      -- anchor
  UNION ALL
  SELECT e.EmpID, e.Name, e.ManagerID, o.Lvl + 1
  FROM Employees e JOIN OrgChart o ON e.ManagerID = o.EmpID -- recursive
)
SELECT * FROM OrgChart ORDER BY Lvl;
```

Q194. What is the difference between a CTE and a subquery?

Aspect	Subquery	CTE
Definition	Nested inline in the query	Declared up front with WITH
Reuse	Must be rewritten if needed again	Referenced multiple times by name
Readability	Harder when deeply nested	Clear, top-down structure
Recursion	Not supported	Supported
Scope	Local to its position	The single following statement

Q195. What is a view?

A: A view is a stored, named SELECT query that behaves like a virtual table. It stores only the query definition, not data, and produces fresh results from the underlying tables each time it is queried. Views simplify access, hide complexity, and add a security layer.

Q196. What is a temporary table?

A: A temporary table is a real table that physically stores data but exists only for the duration of a session (or transaction) and is automatically dropped at the end. It is useful for staging intermediate results that you need to query, index, or reuse across multiple statements.


```
CREATE TEMPORARY TABLE TopCustomers AS
SELECT CustID, SUM(Amount) AS Total
FROM Orders GROUP BY CustID HAVING SUM(Amount) > 10000;
```

Q197. What is the difference between a CTE, a View, and a Temporary Table?

A: This is a very common interview question. The three serve overlapping needs but differ in lifetime, storage, and reusability:

Aspect	CTE	View	Temporary Table
Stores data?	No (logical only)	No (virtual)	Yes (physical)
Lifetime	Single statement	Permanent in schema	Session/transaction
Reusable across queries?	No	Yes	Yes (within session)
Can be indexed?	No	Materialized only	Yes
Recursion?	Yes	No	No
Best for	Readable one-off logic	Reusable saved query	Staging large intermediate data

Q198. When would you choose a temporary table over a CTE?

A: Choose a temporary table when the intermediate result is large, expensive to compute, and used multiple times — because a CTE may be re-evaluated each time it is referenced, while a temp table is computed once and stored. Temp tables can also be indexed and have statistics, helping the optimizer. Use a CTE for readability when the result is used once or is cheap to compute.

Q199. When would you choose a view over a CTE?

A: Choose a view when the same query logic must be reused across many different queries, sessions, or users, since a view is a permanent schema object. A CTE only lives inside a single statement, so it cannot be shared. Views also help centralize security and present a stable interface.

Q200. Is a CTE materialized (computed once) or inlined?

A: It depends on the database. Historically many engines inline a CTE (re-evaluating it wherever it is referenced), while others may materialize it. PostgreSQL materializes by default in older versions but can inline; SQL Server generally inlines. If a CTE is referenced many times and is costly, a temporary table gives more predictable performance.

Q201. What is the difference between a local and a global temporary table (SQL Server)?

A: A local temporary table (prefixed with #) is visible only to the session that created it and is dropped when that session ends. A global temporary table (prefixed with ##) is visible to all sessions and is dropped when the creating session ends and no other session is using it.

Q202. What is the difference between a temporary table and a table variable (SQL Server)?

A: A temporary table is stored in tempdb, can be indexed, holds statistics, and supports larger data and transactions. A table variable is lighter, scoped to a batch/procedure, has limited statistics, and is better for small result sets. Temp tables generally perform better for larger or more complex intermediate data.

Q203. What is the difference between a view and a materialized view (recap in this context)?

A: A normal view re-runs its query every access and always reflects current data but adds computation each time. A materialized view stores the computed result physically for fast reads, trading freshness for speed and requiring periodic refresh. A temporary table is similar to a materialized result but session-scoped and manually managed.

19. Key 'Difference Between' Questions

Comparison questions interviewers love. Each answer states the core distinction crisply.

Q204. Difference between DROP, TRUNCATE, and DELETE?

Feature	DELETE	TRUNCATE	DROP
What it removes	Selected/all rows	All rows	Entire table + structure
WHERE clause	Yes	No	No
Rollback	Yes	Limited	No
Triggers fire	Yes	No	No
Resets identity	No	Yes	N/A
Command type	DML	DDL	DDL
Speed	Slowest	Fast	Fast

Q205. Difference between WHERE and HAVING?

A: WHERE filters individual rows before grouping and cannot use aggregate functions; HAVING filters groups after GROUP BY and is designed to test aggregates like COUNT or SUM. WHERE comes first in execution, HAVING comes after grouping.

Q206. Difference between UNION and UNION ALL?

A: UNION combines result sets and removes duplicate rows, requiring an extra sort/dedupe step. UNION ALL keeps all rows including duplicates and is therefore faster. Use UNION ALL when duplicates are impossible or wanted.

Q207. Difference between JOIN and UNION?

A: A JOIN combines columns from multiple tables side by side based on a related condition (adds width). A UNION stacks rows from multiple queries on top of each other (adds length) and requires matching column counts and types.

Q208. Difference between INNER JOIN and LEFT JOIN?

A: INNER JOIN returns only rows that have matches in both tables. LEFT JOIN returns all rows from the left table plus matching right rows, filling non-matches with NULL. LEFT JOIN preserves unmatched left-side rows; INNER JOIN discards them.

Q209. Difference between Primary Key and Foreign Key?

A: A primary key uniquely identifies each row in its own table and cannot be NULL. A foreign key is a column that references another table's primary key to enforce a relationship; it can contain NULLs and duplicate values. Primary key identifies; foreign key links.

Q210. Difference between Primary Key and Unique Key?

A: Both enforce uniqueness, but a primary key disallows NULLs and a table has exactly one. A unique key allows a single NULL and a table can have many. The primary key is the main identifier; unique keys enforce uniqueness on other columns.

Q211. Difference between CHAR and VARCHAR?

A: CHAR is fixed-length and pads values with spaces to the declared size, ideal for fixed-size codes. VARCHAR is variable-length and stores only the characters used plus a length marker, saving space for values that vary in length.

Q212. Difference between DELETE and DROP?

A: DELETE is a DML command that removes rows while keeping the table structure and can be rolled back. DROP is a DDL command that removes the entire table, including its structure, indexes, and data, and generally cannot be rolled back.

Q213. Difference between a clustered and a non-clustered index?

A: A clustered index determines the physical storage order of the table's rows, so there can be only one per table. A non-clustered index is a separate structure containing a sorted copy of key columns with pointers to the rows, and a table can have many of them.

Q214. Difference between a correlated and a non-correlated subquery?

A: A non-correlated subquery runs independently once and passes its result to the outer query. A correlated subquery references columns from the outer query and is re-evaluated for every outer row, which can be slower but allows row-by-row logic.

Q215. Difference between a function and a stored procedure?

A: A function must return a value and can be used inside a SELECT, but generally cannot modify data. A stored procedure may return zero or more results, can perform INSERT/UPDATE/DELETE and manage transactions, and is invoked with CALL/EXEC rather than inside a query.

Q216. Difference between a stored procedure and a trigger?

A: A stored procedure is invoked explicitly by a user or application. A trigger executes automatically in response to a data event (INSERT/UPDATE/DELETE) on a table and cannot be called directly.

Q217. Difference between TRUNCATE and DELETE in terms of transactions and triggers?

A: DELETE logs each removed row, can use WHERE, fires row-level triggers, and is fully transactional. TRUNCATE deallocates pages with minimal logging, cannot use WHERE, does not fire row-level DELETE triggers, and resets identity counters, making it much faster but less granular.

Q218. Difference between RANK() and DENSE_RANK()?

A: Both give tied rows the same rank, but RANK leaves gaps after ties (1,1,3) while DENSE_RANK does not (1,1,2). Use RANK for competition-style standings with skipped positions and DENSE_RANK when ranks must stay consecutive.

Q219. Difference between ROW_NUMBER() and RANK()?

A: ROW_NUMBER always assigns unique consecutive numbers, breaking ties arbitrarily. RANK gives tied rows the same number and then skips. Use ROW_NUMBER for unique ordering or picking one row per group; use RANK when ties should share a position.

Q220. Difference between COALESCE and ISNULL?

A: COALESCE is ANSI-standard, accepts two or more arguments, and returns the first non-NULL value, working across databases. ISNULL is SQL Server-specific, takes exactly two arguments, and has slightly different data-type handling. Prefer COALESCE for portability and flexibility.

Q221. Difference between scalar and aggregate functions?

A: A scalar function operates on a single value per row and returns one value per row (UPPER, ROUND). An aggregate function operates across many rows and returns a single summarized value per group (SUM, AVG, COUNT).

Q222. Difference between IN and EXISTS?

A: IN compares a value against a list or subquery result set and evaluates the whole set. EXISTS checks whether a (usually correlated) subquery returns any row and stops at the first match. EXISTS is often more efficient for large correlated lookups; IN is simpler for small static lists.

Q223. Difference between a view and a table?

A: A table physically stores data on disk and occupies storage. A view stores only a query definition and produces data on demand from underlying tables. Tables hold data; views present it.

Q224. Difference between normalization and denormalization?

A: Normalization splits data into smaller related tables to reduce redundancy and improve integrity, favoring write consistency. Denormalization deliberately reintroduces redundancy by combining tables to reduce joins and speed up reads, favoring query performance. One optimizes integrity; the other optimizes read speed.

Q225. Difference between DDL and DML?

A: DDL (Data Definition Language) defines or alters database structure (CREATE, ALTER, DROP) and auto-commits. DML (Data Manipulation Language) changes the data within those structures (INSERT, UPDATE, DELETE) and can be rolled back inside a transaction.

Q226. Difference between OLTP and OLAP?

A: OLTP systems handle many small, fast read/write transactions for daily operations and are highly normalized. OLAP systems run complex analytical queries over large historical datasets for reporting and are typically denormalized. OLTP is operational; OLAP is analytical.

PART B — 100+ MOST-ASKED PRACTICAL QUERY QUESTIONS

Each item gives the task and a working query. Table/column names are assumed; adapt to your schema. Dialect is mostly MySQL/standard SQL; notes flag where dialects differ.

A. Basic Retrieval & Filtering

P1. Select all columns from the Employees table.

```
SELECT * FROM Employees;
```

P2. Select only the Name and Salary of all employees.

```
SELECT Name, Salary FROM Employees;
```

P3. Find all employees in department 10.

```
SELECT * FROM Employees WHERE DeptID = 10;
```

P4. Find employees earning more than 50000.

```
SELECT Name, Salary FROM Employees WHERE Salary > 50000;
```

P5. Find employees whose salary is between 40000 and 60000.

```
SELECT Name, Salary FROM Employees  
WHERE Salary BETWEEN 40000 AND 60000;
```

P6. Find employees in departments 10, 20, or 30.

```
SELECT * FROM Employees WHERE DeptID IN (10, 20, 30);
```

P7. Find all employees whose name starts with 'A'.

```
SELECT * FROM Employees WHERE Name LIKE 'A%';
```

P8. Find all employees whose name ends with 'n'.

```
SELECT * FROM Employees WHERE Name LIKE '%n';
```

P9. Find employees whose name contains 'ar' anywhere.

```
SELECT * FROM Employees WHERE Name LIKE '%ar%';
```

P10. Find employees whose name is exactly 5 characters long.

```
SELECT * FROM Employees WHERE Name LIKE '_____'; -- five underscores
```

P11. Find all employees who have no manager assigned.

```
SELECT * FROM Employees WHERE ManagerID IS NULL;
```

P12. List all unique job titles.

```
SELECT DISTINCT JobTitle FROM Employees;
```

P13. List employees sorted by salary in descending order.

```
SELECT Name, Salary FROM Employees ORDER BY Salary DESC;
```

P14. List employees sorted by department, then by salary descending within each department.

```
SELECT Name, DeptID, Salary FROM Employees  
ORDER BY DeptID ASC, Salary DESC;
```

P15. Show the 5 highest-paid employees.

```
SELECT Name, Salary FROM Employees  
ORDER BY Salary DESC LIMIT 5;
```

Note: SQL Server: *SELECT TOP 5 ...* ; Oracle: *... FETCH FIRST 5 ROWS ONLY.*

P16. Show employees not in departments 10 or 20.

```
SELECT * FROM Employees WHERE DeptID NOT IN (10, 20);
```

B. Aggregation & Grouping

P17. Count the total number of employees.

```
SELECT COUNT(*) AS TotalEmployees FROM Employees;
```

P18. Find the total salary paid to all employees.

```
SELECT SUM(Salary) AS TotalSalary FROM Employees;
```

P19. Find the average salary.

```
SELECT AVG(Salary) AS AvgSalary FROM Employees;
```

P20. Find the highest and lowest salary.

```
SELECT MAX(Salary) AS Highest, MIN(Salary) AS Lowest  
FROM Employees;
```

P21. Count employees in each department.

```
SELECT DeptID, COUNT(*) AS EmpCount  
FROM Employees GROUP BY DeptID;
```

P22. Find the average salary in each department.

```
SELECT DeptID, AVG(Salary) AS AvgSalary  
FROM Employees GROUP BY DeptID;
```

P23. Find departments having more than 5 employees.

```
SELECT DeptID, COUNT(*) AS Cnt  
FROM Employees GROUP BY DeptID  
HAVING COUNT(*) > 5;
```

P24. Find the total salary per department, sorted highest first.

```
SELECT DeptID, SUM(Salary) AS TotalSalary  
FROM Employees GROUP BY DeptID  
ORDER BY TotalSalary DESC;
```

P25. Count the number of distinct departments.

```
SELECT COUNT(DISTINCT DeptID) AS DeptCount FROM Employees;
```

P26. Find departments whose average salary exceeds 60000.

```
SELECT DeptID, AVG(Salary) AS AvgSal  
FROM Employees GROUP BY DeptID  
HAVING AVG(Salary) > 60000;
```

P27. Find the number of employees hired each year.


```
SELECT YEAR(HireDate) AS HireYear, COUNT(*) AS Hires
FROM Employees GROUP BY YEAR(HireDate)
ORDER BY HireYear;
```

P28. Find the department with the maximum total salary.

```
SELECT DeptID, SUM(Salary) AS TotalSalary
FROM Employees GROUP BY DeptID
ORDER BY TotalSalary DESC LIMIT 1;
```

C. Classic Interview Queries (Nth highest, duplicates, etc.)

P29. Find the second highest salary.

A: Approach 1 — subquery with MAX:

```
SELECT MAX(Salary) AS SecondHighest
FROM Employees
WHERE Salary < (SELECT MAX(Salary) FROM Employees);
```

Approach 2 — DISTINCT with LIMIT/OFFSET:

```
SELECT DISTINCT Salary FROM Employees
ORDER BY Salary DESC LIMIT 1 OFFSET 1;
```

P30. Find the Nth highest salary (generalized).

```
-- DENSE_RANK handles ties; replace :n with the rank you want
SELECT Salary FROM (
  SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
  FROM Employees
) t WHERE rnk = :n;
```

P31. Find the top 3 salaries (including ties).

```
SELECT Name, Salary FROM (
  SELECT Name, Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
  FROM Employees
) t WHERE rnk <= 3;
```

P32. Find duplicate names in the Employees table.

```
SELECT Name, COUNT(*) AS Occurrences
FROM Employees GROUP BY Name
HAVING COUNT(*) > 1;
```

P33. Find duplicate rows based on Name and DeptID together.

```
SELECT Name, DeptID, COUNT(*) AS Cnt
FROM Employees GROUP BY Name, DeptID
HAVING COUNT(*) > 1;
```

P34. Delete duplicate rows but keep one copy.

```
WITH ranked AS (
  SELECT EmpID,
    ROW_NUMBER() OVER (PARTITION BY Name, DeptID ORDER BY EmpID) AS rn
  FROM Employees
)
DELETE FROM Employees
WHERE EmpID IN (SELECT EmpID FROM ranked WHERE rn > 1);
```

P35. Find employees who earn more than the average salary.

```
SELECT Name, Salary FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

P36. Find the highest-paid employee in each department.

```
SELECT Name, DeptID, Salary FROM (
  SELECT Name, DeptID, Salary,
```



```
RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS rnk  
FROM Employees  
) t WHERE rnk = 1;
```

P37. Find the top 3 earners within each department.

```
SELECT * FROM (  
  SELECT Name, DeptID, Salary,  
    ROW_NUMBER() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS rn  
  FROM Employees  
) t WHERE rn <= 3;
```

P38. Find employees with the same salary as someone else.

```
SELECT Name, Salary FROM Employees  
WHERE Salary IN (  
  SELECT Salary FROM Employees GROUP BY Salary HAVING COUNT(*) > 1  
);
```

P39. Select only odd-numbered rows (by EmpID).

```
SELECT * FROM Employees WHERE MOD(EmpID, 2) = 1;  
-- SQL Server: WHERE EmpID % 2 = 1
```

P40. Swap the values of two columns (e.g., swap 'M' and 'F' in Gender).

```
UPDATE Employees  
SET Gender = CASE WHEN Gender = 'M' THEN 'F'  
  WHEN Gender = 'F' THEN 'M'  
  ELSE Gender END;
```

P41. Find the median salary (works across dialects with window functions).

```
SELECT AVG(Salary) AS MedianSalary FROM (  
  SELECT Salary,  
    ROW_NUMBER() OVER (ORDER BY Salary) AS rn,  
    COUNT(*) OVER () AS cnt  
  FROM Employees  
) t  
WHERE rn IN (FLOOR((cnt+1)/2), CEIL((cnt+1)/2));
```

D. Joins

P42. List each employee with their department name.

```
SELECT e.Name, d.DeptName  
FROM Employees e  
JOIN Departments d ON e.DeptID = d.DeptID;
```

P43. List all departments, including ones with no employees.

```
SELECT d.DeptName, e.Name  
FROM Departments d  
LEFT JOIN Employees e ON d.DeptID = e.DeptID;
```

P44. Find employees who are not assigned to any department.

```
SELECT e.Name  
FROM Employees e  
LEFT JOIN Departments d ON e.DeptID = d.DeptID  
WHERE d.DeptID IS NULL;
```

P45. Find departments that have no employees.

```
SELECT d.DeptName  
FROM Departments d  
LEFT JOIN Employees e ON d.DeptID = e.DeptID  
WHERE e.EmpID IS NULL;
```

P46. List each employee with their manager's name (self join).

```
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.ManagerID = m.EmpID;
```

P47. Count the number of employees in each department name.

```
SELECT d.DeptName, COUNT(e.EmpID) AS EmpCount
FROM Departments d
LEFT JOIN Employees e ON d.DeptID = e.DeptID
GROUP BY d.DeptName;
```

P48. List customers along with their total number of orders.

```
SELECT c.Name, COUNT(o.OrderID) AS TotalOrders
FROM Customers c
LEFT JOIN Orders o ON c.ID = o.CustID
GROUP BY c.Name;
```

P49. Find customers who have never placed an order.

```
SELECT c.Name
FROM Customers c
LEFT JOIN Orders o ON c.ID = o.CustID
WHERE o.OrderID IS NULL;
```

P50. Show every product together with its category and supplier (3-table join).

```
SELECT p.ProductName, c.CategoryName, s.SupplierName
FROM Products p
JOIN Categories c ON p.CategoryID = c.CategoryID
JOIN Suppliers s ON p.SupplierID = s.SupplierID;
```

P51. Find pairs of employees who share the same manager.

```
SELECT a.Name AS Emp1, b.Name AS Emp2, a.ManagerID
FROM Employees a
JOIN Employees b ON a.ManagerID = b.ManagerID AND a.EmpID < b.EmpID;
```

P52. Find total sales amount per customer name.

```
SELECT c.Name, SUM(o.Amount) AS TotalSpent
FROM Customers c
JOIN Orders o ON c.ID = o.CustID
GROUP BY c.Name
ORDER BY TotalSpent DESC;
```

P53. Find the department with the highest average salary.

```
SELECT d.DeptName, AVG(e.Salary) AS AvgSalary
FROM Departments d
JOIN Employees e ON d.DeptID = e.DeptID
GROUP BY d.DeptName
ORDER BY AvgSalary DESC LIMIT 1;
```

E. String & Date Manipulation

P54. Display employee names in uppercase.

```
SELECT UPPER(Name) AS NameUpper FROM Employees;
```

P55. Combine first and last name into a full name.

```
SELECT CONCAT(FirstName, ' ', LastName) AS FullName FROM Employees;
```

P56. Extract the first three characters of each name.

```
SELECT SUBSTRING(Name, 1, 3) AS Prefix FROM Employees;
```

P57. Find the length of each employee's name.

```
SELECT Name, LENGTH(Name) AS NameLength FROM Employees;
-- SQL Server: LEN(Name)
```

P58. Replace all occurrences of 'St.' with 'Street' in addresses.

```
SELECT REPLACE(Address, 'St.', 'Street') AS CleanAddress FROM Customers;
```

P59. Remove leading and trailing spaces from a column.

```
SELECT TRIM(Name) AS Cleaned FROM Employees;
```

P60. Get the current date and time.

```
SELECT NOW();    -- MySQL
SELECT GETDATE(); -- SQL Server
SELECT SYSDATE FROM dual; -- Oracle
```

P61. Find employees hired in the year 2023.

```
SELECT * FROM Employees WHERE YEAR(HireDate) = 2023;
```

P62. Calculate each employee's tenure in years.

```
SELECT Name,
TIMESTAMPDIFF(YEAR, HireDate, CURDATE()) AS Years
FROM Employees; -- MySQL
```

P63. Find employees hired in the last 30 days.

```
SELECT * FROM Employees
WHERE HireDate >= DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```

P64. Extract the month name from a hire date.

```
SELECT Name, MONTHNAME(HireDate) AS HireMonth FROM Employees;
```

P65. Mask an email so only the first letter and domain show.

```
SELECT CONCAT(LEFT(Email,1), '***', SUBSTRING(Email, INSTR(Email,'@')))
AS MaskedEmail
FROM Customers;
```

F. DDL & DML (Create, Insert, Update, Delete)

P66. Create an Employees table with basic columns and constraints.

```
CREATE TABLE Employees (
EmpID INT PRIMARY KEY,
Name VARCHAR(50) NOT NULL,
DeptID INT,
Salary DECIMAL(10,2) CHECK (Salary > 0),
HireDate DATE DEFAULT (CURRENT_DATE)
);
```

P67. Add a new column 'Email' to the Employees table.

```
ALTER TABLE Employees ADD Email VARCHAR(100);
```

P68. Change the data type of the Salary column.

```
ALTER TABLE Employees MODIFY Salary DECIMAL(12,2);
-- SQL Server: ALTER COLUMN Salary DECIMAL(12,2)
```

P69. Drop the Email column.

```
ALTER TABLE Employees DROP COLUMN Email;
```

P70. Rename the Employees table to Staff.

```
ALTER TABLE Employees RENAME TO Staff;
-- SQL Server: EXEC sp_rename 'Employees', 'Staff';
```

P71. Insert a new employee record.

```
INSERT INTO Employees (EmpID, Name, DeptID, Salary)
VALUES (101, 'Ravi', 10, 55000);
```

P72. Insert multiple rows in a single statement.

```
INSERT INTO Employees (EmpID, Name, DeptID, Salary) VALUES
(102, 'Anita', 20, 62000),
(103, 'Sameer', 10, 48000);
```

P73. Give every employee in department 10 a 10% raise.

```
UPDATE Employees
SET Salary = Salary * 1.10
WHERE DeptID = 10;
```

P74. Set the DeptID to 99 for employees who currently have none.

```
UPDATE Employees SET DeptID = 99 WHERE DeptID IS NULL;
```

P75. Delete employees who earn less than 20000.

```
DELETE FROM Employees WHERE Salary < 20000;
```

P76. Remove all rows from a table quickly without dropping it.

```
TRUNCATE TABLE TempData;
```

P77. Copy all rows from Employees into an Employees_Backup table.

```
INSERT INTO Employees_Backup
SELECT * FROM Employees;
```

```
-- Or create + copy in one step:
CREATE TABLE Employees_Backup AS SELECT * FROM Employees;
```

P78. Update a column using a value from another table (correlated update).

```
UPDATE Employees e
JOIN Departments d ON e.DeptID = d.DeptID
SET e.DeptName = d.DeptName; -- MySQL multi-table update
```

G. Window Functions & Analytics

P79. Rank employees by salary (highest = rank 1).

```
SELECT Name, Salary,
       RANK() OVER (ORDER BY Salary DESC) AS SalaryRank
FROM Employees;
```

P80. Assign a unique row number to each employee ordered by hire date.

```
SELECT Name, HireDate,
       ROW_NUMBER() OVER (ORDER BY HireDate) AS RowNum
FROM Employees;
```

P81. Rank employees within their department by salary.

```
SELECT Name, DeptID, Salary,
       DENSE_RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS DeptRank
FROM Employees;
```

P82. Show each employee's salary alongside the department average.

```
SELECT Name, DeptID, Salary,
       AVG(Salary) OVER (PARTITION BY DeptID) AS DeptAvg
FROM Employees;
```

P83. Compute a running total of order amounts by date.

```
SELECT OrderDate, Amount,
       SUM(Amount) OVER (ORDER BY OrderDate) AS RunningTotal
FROM Orders;
```

P84. Show each month's sales and the previous month's sales.

```
SELECT Month, Sales,
       LAG(Sales) OVER (ORDER BY Month) AS PrevMonth
FROM MonthlySales;
```

P85. Calculate month-over-month sales growth.

```
SELECT Month, Sales,  
       Sales - LAG(Sales) OVER (ORDER BY Month) AS Growth  
FROM MonthlySales;
```

P86. Divide customers into 4 spending quartiles.

```
SELECT Name, TotalSpent,  
       NTILE(4) OVER (ORDER BY TotalSpent DESC) AS Quartile  
FROM CustomerSpend;
```

P87. Find the first order date for each customer.

```
SELECT CustID, OrderDate,  
       FIRST_VALUE(OrderDate) OVER  
       (PARTITION BY CustID ORDER BY OrderDate) AS FirstOrder  
FROM Orders;
```

P88. Find each employee's salary as a percentage of the department total.

```
SELECT Name, DeptID, Salary,  
       ROUND(100.0 * Salary /  
             SUM(Salary) OVER (PARTITION BY DeptID), 2) AS PctOfDept  
FROM Employees;
```

H. Subqueries, Set Operations & Conditional Logic

P89. Find employees who earn more than the company average.

```
SELECT Name, Salary FROM Employees  
WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

P90. Find employees who work in the same department as 'Ravi'.

```
SELECT Name FROM Employees  
WHERE DeptID = (SELECT DeptID FROM Employees WHERE Name = 'Ravi')  
AND Name <> 'Ravi';
```

P91. Find departments where the maximum salary exceeds 90000.

```
SELECT DeptID FROM Employees  
GROUP BY DeptID  
HAVING MAX(Salary) > 90000;
```

P92. Use EXISTS to find customers who placed at least one order.

```
SELECT Name FROM Customers c  
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.CustID = c.ID);
```

P93. Combine active customers from two regions (remove duplicates).

```
SELECT Name FROM NorthCustomers  
UNION  
SELECT Name FROM SouthCustomers;
```

P94. Find customers present in both the 2023 and 2024 lists.

```
SELECT CustID FROM Orders2023  
INTERSECT  
SELECT CustID FROM Orders2024;
```

P95. Find customers who ordered in 2023 but not in 2024.

```
SELECT CustID FROM Orders2023  
EXCEPT  
SELECT CustID FROM Orders2024; -- Oracle: MINUS
```

P96. Label salaries as High / Medium / Low using CASE.

```
SELECT Name, Salary,  
       CASE WHEN Salary >= 80000 THEN 'High'  
            WHEN Salary >= 50000 THEN 'Medium'
```



```
ELSE 'Low' END AS Band  
FROM Employees;
```

P97. Count how many employees fall into each salary band.

```
SELECT  
SUM(CASE WHEN Salary >= 80000 THEN 1 ELSE 0 END) AS High,  
SUM(CASE WHEN Salary BETWEEN 50000 AND 79999 THEN 1 ELSE 0 END) AS Medium,  
SUM(CASE WHEN Salary < 50000 THEN 1 ELSE 0 END) AS Low  
FROM Employees;
```

P98. Pivot: show total sales per region as columns.

```
SELECT  
SUM(CASE WHEN Region='North' THEN Amount END) AS North,  
SUM(CASE WHEN Region='South' THEN Amount END) AS South,  
SUM(CASE WHEN Region='East' THEN Amount END) AS East  
FROM Sales;
```

P99. Find the department with the most employees using a subquery.

```
SELECT DeptID FROM Employees  
GROUP BY DeptID  
HAVING COUNT(*) = (  
    SELECT MAX(c) FROM (  
        SELECT COUNT(*) AS c FROM Employees GROUP BY DeptID  
    ) t  
);
```

P100. Replace NULL commissions with 0 in output.

```
SELECT Name, COALESCE(Commission, 0) AS Commission  
FROM Employees;
```

P101. Show employees whose salary is above their own department's average.

```
SELECT Name, DeptID, Salary FROM Employees e  
WHERE Salary > (  
    SELECT AVG(Salary) FROM Employees x WHERE x.DeptID = e.DeptID  
);
```

P102. Find the 3rd, 4th and 5th highest salaries.

```
SELECT DISTINCT Salary FROM Employees  
ORDER BY Salary DESC LIMIT 3 OFFSET 2;
```

P103. List employees and flag whether they earn above or below 60000.

```
SELECT Name, Salary,  
    IF(Salary > 60000, 'Above', 'Below') AS Flag  
FROM Employees; -- standard: CASE WHEN ...
```

Practice every query in a real SQL engine. Reading answers builds recognition; typing them builds recall — and interviews test recall.